Lab Assignment 16.1

1. Student Information System Schema
   Task:
   Design a database schema for a Student Information System and generate queries using AI.
   Instructions:
   - Tables: Students, Courses, Enrollments.
   - Define primary keys, foreign keys, and relationships.
   - Generate queries:
     - Insert a new student record.
     - Fetch all courses enrolled by a student.
     - Count the number of students in each course.

   Expected Output:
   SQL CREATE TABLE statements and queries for student–course relationships.

Screenshots:

ASSG-16-1.sql

```sql
--create tables named as students and courses and enrollments  and define  primary keys foreign keys and realtionships
CREATE TABLE IF NOT EXISTS courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL
);
CREATE TABLE IF NOT EXISTS enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
CREATE TABLE IF NOT EXISTS students (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT NOT NULL,
    grade VARCHAR(10) NOT NULL
);
```

```sql
--insert sample data into courses table
INSERT INTO courses (course_id, course_name) VALUES
(1, 'Mathematics'),
(2, 'Physics'),
(3, 'Chemistry');
--insert sample data into enrollments table
INSERT INTO enrollments (enrollment_id, student_id, course_id) VALUES
(1, 1, 1),
(2, 1, 2),
(3, 2, 1),
(4, 3, 3),
(5, 4, 2);
--insert sample data into students table
INSERT INTO students (id, name, age, grade) VALUES
(1, 'Alice', 20, 'A'),
(2, 'Bob', 22, 'B'),
(3, 'Charlie', 19, 'A'),
(4, 'David', 21, 'C'),
(5, 'Eve', 20, 'B');
--display all rows form the students table
```

Output:

SQL ▾ 1 / 1 1 - 5 of 5

student_name	course_name
Alice	Mathematics
Alice	Physics
Bob	Mathematics
Charlie	Chemistry
David	Physics

SQL ▾ 1 / 1 1 - 3 of 3

course_name	student_count
Chemistry	1
Mathematics	2
Physics	2

2. Hospital Management Database
   Task:
   Create schema and queries for a Hospital Management System.
   Instructions:
   - Tables: Doctors, Patients, Appointments.
   - Use AI to define constraints (unique IDs, valid dates).
   - Generate queries:
     - List all appointments for a specific doctor.
     - Retrieve patient history by patient ID.
     - Count total patients treated by each doctor.

   Expected Output:
   - Normalized schema and SQL queries with joins.

Screenshots:

```sql
--create tables named as doctors and patients appointments and define  primary keys foreign keys and realtionships contra
CREATE TABLE IF NOT EXISTS doctors (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    specialty VARCHAR(50) NOT NULL
);
CREATE TABLE IF NOT EXISTS patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT NOT NULL
);
CREATE TABLE IF NOT EXISTS appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
);
```

```sql
--insert sample data into doctors table
INSERT INTO doctors (doctor_id, name, specialty) VALUES
(1, 'Dr. Smith', 'Cardiology'),
(2, 'Dr. Johnson', 'Neurology'),
:\Users\arell\Music\aiac\assg_01.py ediatrics'),
(4, 'Dr. Brown', 'Dermatology'),
(5, 'Dr. Davis', 'Orthopedics');
--insert sample data into patients table
INSERT INTO patients (patient_id, name, age) VALUES
(1, 'Alice', 30),
(2, 'Bob', 25),
(3, 'Charlie', 35),
(4, 'David', 28),
(5, 'Eve', 22);
--insert sample data into appointments table
INSERT INTO appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
(1, 1, 1, '2024-07-01'),
(2, 2, 2, '2024-07-02'),
(3, 3, 3, '2024-07-03'),
(4, 4, 4, '2024-07-04'),
(5, 5, 5, '2024-07-05');
```

```sql
--list all appointments for a specific doctor
SELECT d.name AS doctor_name, p.name AS patient_name, a.appointment_date
FROM appointments a
JOIN doctors d ON a.doctor_id = d.doctor_id
JOIN patients p ON a.patient_id = p.patient_id
WHERE d.name = 'Dr. Smith';
--retrive patients history by patient id
SELECT p.name AS patient_name, d.name AS doctor_name, a.appointment_date
FROM appointments a
JOIN doctors d ON a.doctor_id = d.doctor_id
JOIN patients p ON a.patient_id = p.patient_id
WHERE p.patient_id = 1;
--count total patients treated by each doctor
SELECT d.name AS doctor_name, COUNT(a.patient_id) AS patient_count
FROM doctors d
LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name;
```

SQL ▼ 1 / 1 1 - 1 of 1

doctor_name	patient_name	appointment_date
Dr. Smith	Alice	2024-07-01

SQL ▼ 1 / 1 1 - 1 of 1

patient_name	doctor_name	appointment_date
Alice	Dr. Smith	2024-07-01

SQL ▼ 1 / 1 1 - 5 of 5

doctor_name	patient_count
Dr. Brown	1
Dr. Davis	1
Dr. Johnson	1
Dr. Lee	1
Dr. Smith	1

3. Library Database
   Task:
   Design schema for a Library Management System.
   Instructions:
   - Tables: Books, Members, Loans.
   - Use AI to suggest indexing strategy for faster queries.
   - Generate queries:
     - Retrieve all books currently issued.
     - Find overdue books (loan date > 30 days).
     - Count the number of books loaned by each member.

   Expected Output:
   - Schema + SQL queries demonstrating joins and conditions.

Screenshots:

```sql
--create tables books ,members ,loans use indexxing stratagies for faster queries
CREATE TABLE IF NOT EXISTS books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(50) NOT NULL
);
CREATE TABLE IF NOT EXISTS members (
    member_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    membership_date DATE NOT NULL
);
CREATE TABLE IF NOT EXISTS loans (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books(book_id),
    FOREIGN KEY (member_id) REFERENCES members(member_id)
);
--insert sample data into books table
```

```sql
--insert sample data into all tables
INSERT INTO books (book_id, title, author) VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald'),
(2, 'To Kill a Mockingbird', 'Harper Lee'),
(3, '1984', 'George Orwell'),
(4, 'Pride and Prejudice', 'Jane Austen'),
(5, 'Moby Dick', 'Herman Melville');
INSERT INTO members (member_id, name, membership_date) VALUES
(1, 'Alice', '2024-01-01'),
(2, 'Bob', '2024-01-02'),
(3, 'Charlie', '2024-01-03'),
(4, 'David', '2024-01-04'),
(5, 'Eve', '2024-01-05');
INSERT INTO loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 1, 1, '2024-07-01', NULL),
(2, 2, 2, '2024-07-02', NULL),
(3, 3, 3, '2024-07-03', NULL),
(4, 4, 4, '2024-07-04', NULL),
(5, 5, 5, '2024-07-05', NULL);
```

```sql
--retrive all books currently issued
SELECT b.title, b.author, m.name AS member_name
FROM loans l
JOIN books b ON l.book_id = b.book_id
JOIN members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
--find overdue books (load date>30 days )
SELECT b.title, b.author, m.name AS member_name
FROM loans l
JOIN books b ON l.book_id = b.book_id
JOIN members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL AND DATEDIFF(CURDATE(), l.loan_date) > 30;
--count no of books loaned by each member
SELECT m.name AS member_name, COUNT(l.book_id) AS books_loaned
FROM members m
LEFT JOIN loans l ON m.member_id = l.member_id
GROUP BY m.name;
```

Output:

SQL ▼ 1 / 1 1 - 5 of 5

title	author	member_name
The Great Gatsby	F. Scott Fitzgerald	Alice
To Kill a Mockingbird	Harper Lee	Bob
1984	George Orwell	Charlie
Pride and Prejudice	Jane Austen	David
Moby Dick	Herman Melville	Eve

SQL ▼ 1 / 1 1 - 5 of 5

title	author	member_name
The Great Gatsby	F. Scott Fitzgerald	Alice
To Kill a Mockingbird	Harper Lee	Bob
1984	George Orwell	Charlie
Pride and Prejudice	Jane Austen	David
Moby Dick	Herman Melville	Eve

SQL ▼ 1 / 1 1 - 5 of 5

member_name	books_loaned
Alice	1
Bob	1
Charlie	1
David	1
Eve	1

4. Real-Time Application: Online Shopping Database
   Scenario:
   Design a database for an E-commerce platform.
   Requirements:
   - Tables: Users, Products, Orders, OrderDetails.
   - Generate queries to:
     - Retrieve all orders by a user.
     - Find the most purchased product.
     - Calculate total revenue in a given month.
   - AI should also suggest normalization improvements and query optimization.

   Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
  - Retrieve all orders by a user.
  - Find the most purchased product.
  - Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

```sql
--create tables users,products,orders,orderdetails use normalization improvements and query optimization techniques
CREATE TABLE IF NOT EXISTS users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL
);
CREATE TABLE IF NOT EXISTS products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10, 2) NOT NULL
);
CREATE TABLE IF NOT EXISTS orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    order_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
CREATE TABLE IF NOT EXISTS order_details (
    order_detail_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
--insert sample data into all tables
```

```sql
--insert sample data into all tables
INSERT INTO users (user_id, username, email, password) VALUES
(1, 'alice', 'alice@example.com', 'password1'),
(2, 'bob', 'bob@example.com', 'password2'),
(3, 'charlie', 'charlie@example.com', 'password3');
INSERT INTO products (product_id, product_name, price) VALUES
(1, 'Laptop', 999.99),
(2, 'Smartphone', 499.99),
(3, 'Headphones', 199.99);
INSERT INTO orders (order_id, user_id, order_date) VALUES
(1, 1, '2024-07-01'),
(2, 2, '2024-07-02'),
(3, 3, '2024-07-03');
INSERT INTO order_details (order_detail_id, order_id, product_id, quantity) VALUES
(1, 1, 1, 1),
(2, 1, 3, 2),
(3, 2, 2, 1),
(4, 3, 1, 1),
(5, 3, 2, 2);
```

```sql
--retrive all orders by a user
SELECT u.username, o.order_date, p.product_name, od.quantity
FROM orders o
JOIN users u ON o.user_id = u.user_id
JOIN order_details od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id;
--find the most purchased product
SELECT p.product_name, SUM(od.quantity) AS total_quantity
FROM order_details od
JOIN products p ON od.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_quantity DESC
LIMIT 1;
--calculate total revenue in a given month
SELECT strftime('%Y-%m', o.order_date) AS month, SUM(p.price * od.quantity) AS total_revenue
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
WHERE o.order_date >= '2024-07-01' AND o.order_date < '2024-08-01'
GROUP BY month;
```

Output:

SQL ▼ 1 / 1 1 - 5 of 5

username	order_date	product_name	quantity
alice	2024-07-01	Laptop	1
alice	2024-07-01	Headphones	2
bob	2024-07-02	Smartphone	1
charlie	2024-07-03	Laptop	1
charlie	2024-07-03	Smartphone	2

SQL ▼ 1 / 1 1 - 1 of 1

product_name	total_quantity
Smartphone	3

SQL ▼ 1 / 1 1 - 1 of 1

month	total_revenue
2024-07	3899.93

5. University Examination Database
   Design a database schema for a University Examination System and generate SQL queries using AI.
   Instructions:
   - Tables: Students, Subjects, Exams, Results.
   - Define primary keys, foreign keys, and referential integrity constraints.
   - Generate queries:
     - Insert examination results for a student.
     - Retrieve subject-wise marks for a student.
     - Calculate average marks for each subject.

   Expected Output:
   - Normalized schema and SQL queries involving aggregation and joins.

Screenshots:

```sql
--create tables named as students  and subjects and exams and results and Define primary keys, foreign keys, and referent
CREATE TABLE IF NOT EXISTS students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT NOT NULL
);
CREATE TABLE IF NOT EXISTS subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(50) NOT NULL
);
CREATE TABLE IF NOT EXISTS exams (
    exam_id INT PRIMARY KEY,
    subject_id INT,
    exam_date DATE NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
);
CREATE TABLE IF NOT EXISTS results (
    result_id INT PRIMARY KEY,
    student_id INT,
    exam_id INT,
    score DECIMAL(5, 2) NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (exam_id) REFERENCES exams(exam_id)
);
```

```sql
--insert sample data into all tables
INSERT INTO students (student_id, name, age) VALUES
(1, 'Alice', 20),
(2, 'Bob', 22),
(3, 'Charlie', 19),
(4, 'David', 21),
(5, 'Eve', 20);
INSERT INTO subjects (subject_id, subject_name) VALUES
(1, 'Mathematics'),
(2, 'Physics'),
(3, 'Chemistry'),
(4, 'Biology'),
(5, 'History');
INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
(1, 1, '2024-07-01'),
(2, 2, '2024-07-02'),
(3, 3, '2024-07-03'),
(4, 4, '2024-07-04'),
(5, 5, '2024-07-05');
```

```sql
--retrive  subject-wise marks for a student
SELECT s.name AS student_name, sub.subject_name, r.score
FROM results r
JOIN students s ON r.student_id = s.student_id
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
WHERE s.student_id = 1;
--calculate avg marks for each subject
SELECT sub.subject_name, AVG(r.score) AS average_score
FROM results r
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
GROUP BY sub.subject_name;
```

Output:

SQL ▼ 1 / 1 1 - 2 of 2

student_name	subject_name	score
Alice	Mathematics	88.5
Alice	Physics	91

SQL ▼ 1 / 1 1 - 5 of 5

subject_name	average_score
Biology	79.0
Chemistry	84.5
History	92.0
Mathematics	82.25
Physics	91.0